

Engineering Security Features Without Engineering a Privacy Incident

Raw security logs are not useful to a machine learning model on their own; a single login event tells you almost nothing, but a user's login pattern over the last hour, compared to their usual pattern, tells you a lot. Feature engineering is the work of turning raw telemetry into these comparative, aggregated signals a model can actually learn from. But every feature that makes a model smarter about a user's behavior is also, by definition, a feature that describes a real person's behavior in more detail. This report documents twelve security features across four categories, and for each one asks not just "does this catch an attack" but "what does this reveal about a person, and how do we limit that."

Temporal Features

1. Login failure count per user per hour

Definition: count(status = FAILURE) grouped by user, grouped by hour

Detects: Catches brute-force and password-spraying attempts, where an attacker tries many passwords against one account in a short window. A sudden jump from a user's normal 0-1 failures to 15+ in an hour is a strong signal.

Data sources: Authentication logs (timestamp, user, status).

Privacy risk: Low

Mitigation: The count alone doesn't reveal much beyond login behavior; still worth aggregating at the hour level rather than storing every individual timestamped failure, to limit precision.

2. Time since last login (abnormal-hour detection)

Definition: current_login_timestamp - previous_login_timestamp for the same user, in minutes

Detects: Flags logins at unusual times for that specific user, e.g., an account that always logs in 9am-6pm suddenly logging in at 3am, which can indicate a compromised credential being used by someone in a different time zone.

Data sources: Authentication logs (timestamp, user).

Privacy risk: Medium

Mitigation: This feature can reveal a person's routine and lifestyle (when they sleep, when they travel). Store only the derived minutes-since-last-login value, not a running log of every login time, and restrict access to the underlying timestamps.

3. Event velocity (events per minute)

Definition: count(events) / time_window_minutes, per user or per host

Detects: Detects automated or scripted activity, such as a compromised account or bot rapidly generating requests far faster than a human could type or click.

Data sources: Any event log with a timestamp (auth, network, or DNS).

Privacy risk: Low

Mitigation: Aggregate counts don't expose specific actions taken, so this is low risk on its own; still should be tied to retention limits like the other temporal features.

Behavioral Features

4. Failed-to-successful login ratio

Definition: failure_count / (success_count + failure_count) per user per window

Detects: A high ratio suggests repeated guessing rather than a person occasionally mistyping their password; useful for separating brute-force attempts from normal human error.

Data sources: Authentication logs (status).

Privacy risk: Low

Mitigation: A ratio is inherently less revealing than raw counts since it's normalized; safe to store at the aggregate level without further controls.

5. Unique destination IPs per source

Definition: count(distinct dest_ip) grouped by source_ip per window

Detects: Catches network scanning or reconnaissance, where a compromised host or attacker probes many different destinations in a short time instead of the small, stable set of destinations a normal workstation talks to.

Data sources: Network/firewall logs (source_ip, dest_ip).

Privacy risk: Medium

Mitigation: IP addresses can be tied to physical locations or specific devices tied to a person. Pseudonymize the source_ip in any feature output that leaves the security team, and keep the raw IP-to-identity mapping in a separate, access-controlled store.

6. DNS query entropy (DGA detection)

Definition: Shannon entropy of the characters in the queried domain name: $H = -\sum(p_i * \log_2(p_i))$ over each character's frequency p_i

Detects: Domain Generation Algorithms used by malware produce random-looking domains (e.g., 'xkq83jfoa.com') that have much higher character entropy than normal human-chosen domains, so a sudden spike in high-entropy queries from a host is a strong malware indicator.

Data sources: DNS logs (query_domain).

Privacy risk: Low

Mitigation: The entropy score itself doesn't reveal browsing content; still, avoid storing the full domain string alongside the score in shared dashboards where it isn't operationally necessary.

Aggregational Features

7. Peer group deviation (user vs. team average)

Definition: $z = (\text{user_value} - \text{team_mean}) / \text{team_std_dev}$, for a chosen metric like login count or bytes transferred

Detects: Flags a user behaving very differently from their peers doing the same job, e.g., a finance analyst suddenly accessing far more systems than the rest of the finance team, which can indicate a compromised account or insider activity.

Data sources: Authentication and network logs, plus an org-chart or team mapping.

Privacy risk: High

Mitigation: This directly compares one named individual against colleagues, which is sensitive and can feel like surveillance if misused. Compute deviations on pseudonymized IDs, require a documented business justification before deviation scores are used for any HR-adjacent decision, and never expose peer-comparison scores outside the security team.

8. Data transfer volume per session

Definition: sum(bytes) grouped by session (user or host) per window

Detects: Detects data exfiltration, where an attacker or malicious insider moves an unusually large volume of data out of the network compared to that user's normal transfer volume.

Data sources: Network logs (bytes, source_ip, timestamp).

Privacy risk: Medium

Mitigation: Transfer volume alone doesn't reveal content, but combined with destination and timing it can suggest what a person was doing. Store aggregated volume without retaining full packet-level detail beyond what's needed for the alert window.

Derived Features

9. Geographic distance between consecutive logins

Definition: haversine_distance(login_location_n, login_location_n-1) combined with elapsed time, to flag physically impossible travel

Detects: Classic 'impossible travel' detection: if a user logs in from New York and then, twenty minutes later, from Singapore, that is physically impossible and strongly suggests a stolen credential being used from two places at once.

Data sources: Authentication logs (source_ip, timestamp) plus IP-to-geolocation lookup.

Privacy risk: High

Mitigation: This is one of the most privacy-sensitive features on this list since it tracks a person's physical location over time. Store only the derived flag (impossible/possible) and the distance/speed value, not a running location history, and restrict raw geolocation data to a short retention window.

10. Rare process execution frequency

Definition: count(this exact process/hash executed org-wide) over a trailing window; low counts flagged as rare

Detects: Surfaces processes that almost nobody else in the organization runs, which is a common signal for custom malware or an attacker's tooling, since attacker tools are typically far less common than everyday business software.

Data sources: Endpoint/host logs (process name or hash, host, timestamp).

Privacy risk: Low

Mitigation: Process names generally describe software, not personal behavior; still worth excluding personal or user-installed software names that could reveal sensitive interests (e.g., health or job-search apps) from any shared dashboard.

Privacy-Aware Feature Engineering

Pseudonymization: instead of storing a user's real username or an IP address directly in a feature table, replace it with a one-way hash (e.g., HMAC-SHA256 with a secret key known only to the security team). This lets the same person's events be linked together across a feature matrix for pattern detection, without the feature file itself containing a directly identifying username. The mapping from hash back to real identity is kept in a separate, access-controlled store, so a data scientist working with the feature matrix never needs to see real usernames to build or run a model.

k-anonymity: for any aggregation broken down by a small group (e.g., peer-group deviation by team), a group of size 1 or 2 makes it easy to reverse-identify the person even from a pseudonymized ID. A basic k-anonymity rule is to suppress or generalize any aggregate computed over a group smaller than k (commonly k=5): if a 'team' has only two members, either merge it with a related team for the purpose of the deviation calculation, or withhold the peer-deviation feature for that group entirely.

Differential privacy: for features that will be shared more broadly (e.g., in a cross-team dashboard or a published research dataset), adding a small amount of calibrated statistical noise to aggregate counts or averages ensures that no single person's presence or absence in the dataset measurably changes the output, while the aggregate statistic as a whole stays useful. This is a heavier-weight control best reserved for features leaving the security team's own access-

controlled environment, since it does reduce precision, which matters less for a dashboard and more for a live detection feature.

Feature Selection: Explainability vs. Performance

Not every feature that improves a model's accuracy score belongs in a production security pipeline. A feature should be included when an analyst can look at a flagged alert and explain, in one sentence, why that feature made the alert fire — 'this account had 20 failed logins in an hour' is immediately actionable, while an opaque combination of a dozen correlated features that only a model can interpret is much harder for a human analyst to act on, verify, or defend in an incident report. A feature should be excluded, or held to extra scrutiny, when its privacy risk is high (like peer-group deviation or geolocation) and its marginal gain in detection accuracy over simpler, lower-risk features is small. As a rule of thumb: start with low-risk, high-explainability features (counts, ratios, entropy) as the backbone of a detection model, and add higher-risk, harder-to-explain features (peer deviation, geolocation) only when they demonstrably catch attacks the simpler features miss, and only with the mitigations described above already in place.

AI Assistance Note

I used Claude as an AI assistant for this assignment. For Task 1, I used it to draft the Feature Engineering Brief, including the feature definitions, detection rationale, and privacy risk assessments for the 10 required features, based on the assignment's category requirements; I reviewed the reasoning and content myself. For Task 2, I used it to help design and implement the feature extraction pipeline (entity resolution logic, feature calculations, pseudonymization approach, and CLI), and to generate the synthetic sample event data used to test it. For Task 3, I used it to help design and implement the data quality and drift validation logic (schema/null/range/duplicate checks and the KS-test-based drift detection), and to write the accompanying documentation (feature_dictionary.md, drift_analysis.md, README.md). I also used it to help build and execute the exploratory data analysis notebook (EDA.ipynb). I verified that the scripts run correctly against the sample data and reviewed the outputs before submitting.